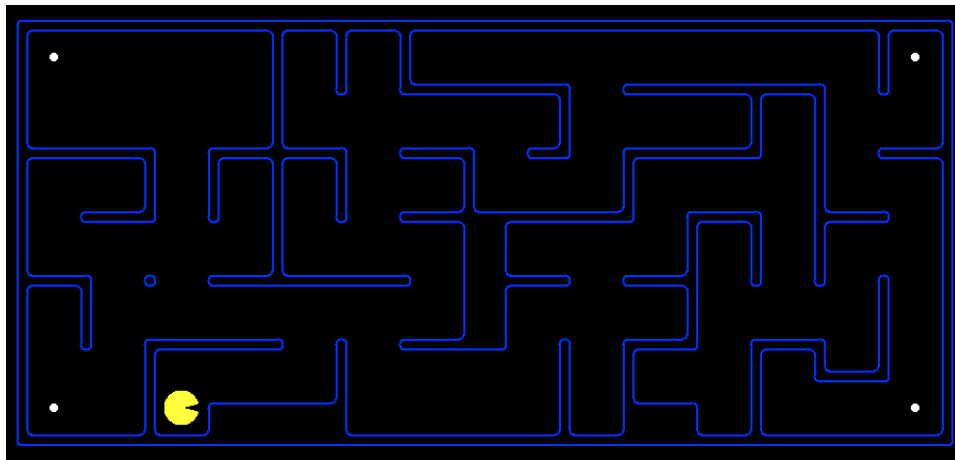


SI3003 - Inteligencia Artificial

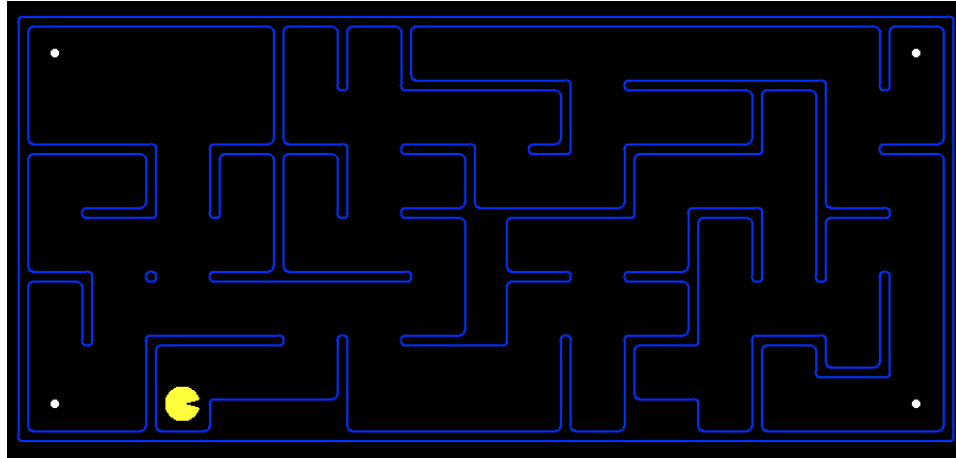
CLASE 3 – SATISFACCIÓN DE RESTRICCIONES (CSP)

EAFIT

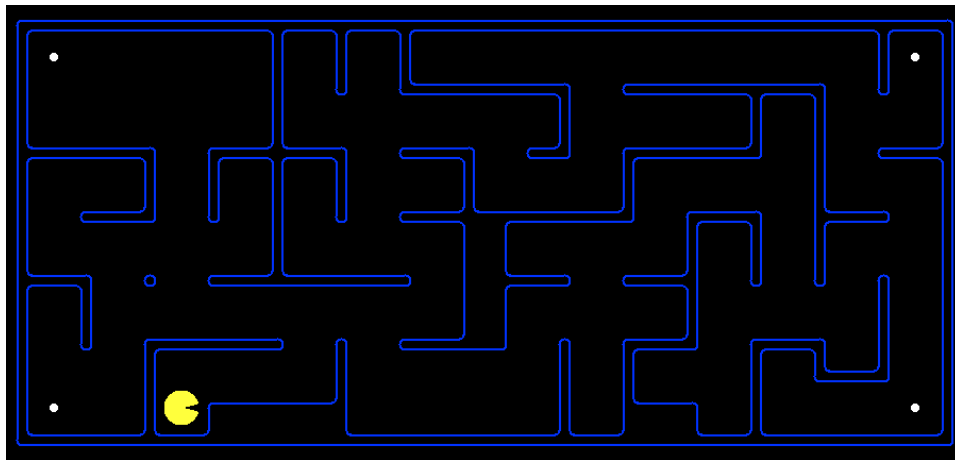
UNIVERSIDAD



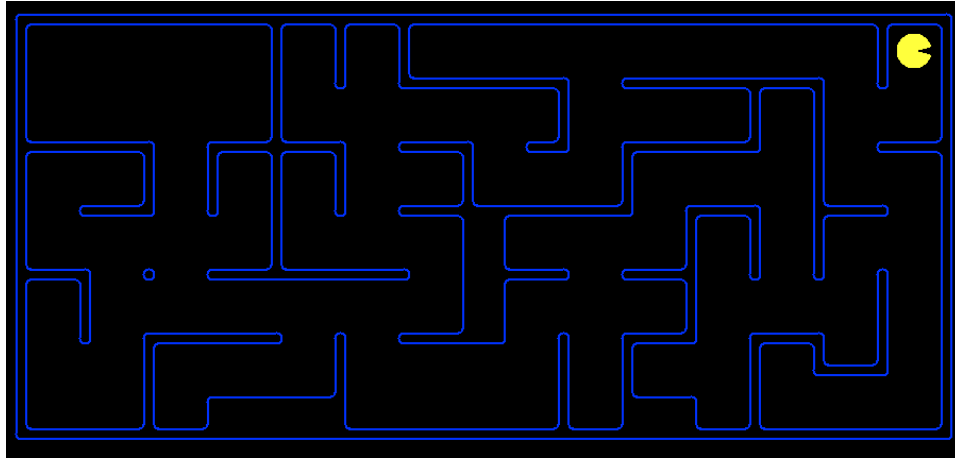
Mmm, déjame pensar...



(...)



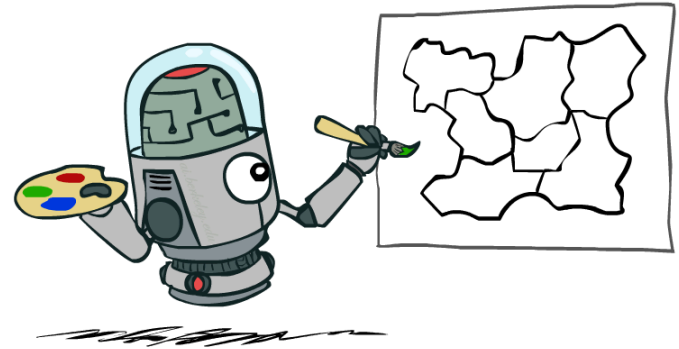
(un rato después)



¡Solución encontrada! ¿Podemos hacerlo mejor?

Hoy

- *Problemas de satisfacción de restricciones (CSP):*
 - Explotar la representación de un estado para acelerar la búsqueda.
 - Backtracking.
 - Heurísticas genéricas: ordenamiento, filtrado, estructura.



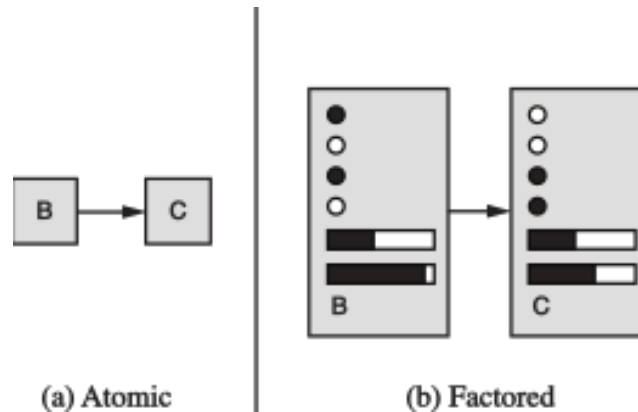
Problemas de satisfacción de restricciones

Motivación

En los problemas de búsqueda estándar:

- Los estados se evalúan con heurísticas específicas del dominio.
- Los estados se prueban con una función específica del dominio para determinar si se alcanzó el objetivo.
- Sin embargo, desde el punto de vista de los algoritmos de búsqueda, **los estados son atómicos**.

En cambio, si los estados tienen una *representación factored*, entonces se puede explotar la estructura de los estados para mejorar la **eficiencia de la búsqueda**.



Problemas de satisfacción de restricciones

- Los solucionadores de

problemas de satisfacción de restricciones

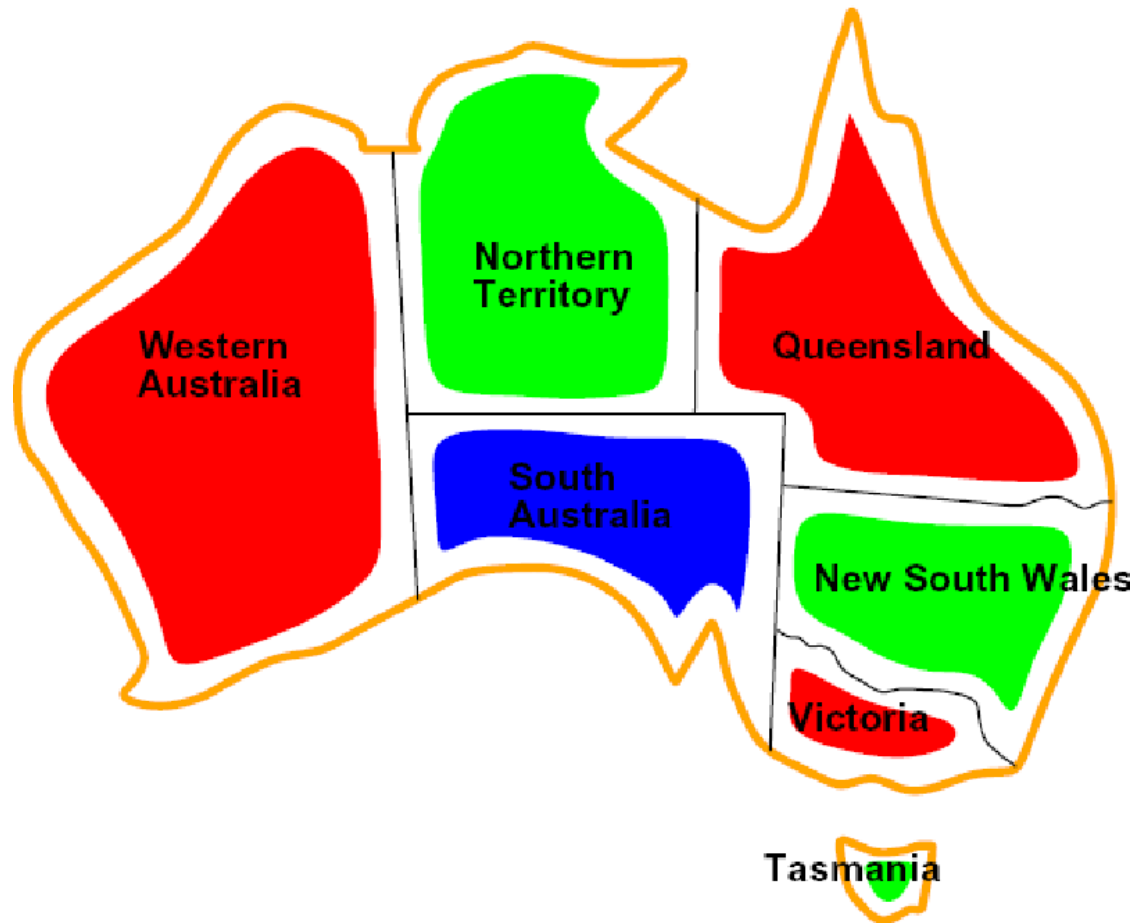
(CSP) aprovechan las representaciones factored de estados y usan heurísticas de *propósito general* para resolver problemas complejos.

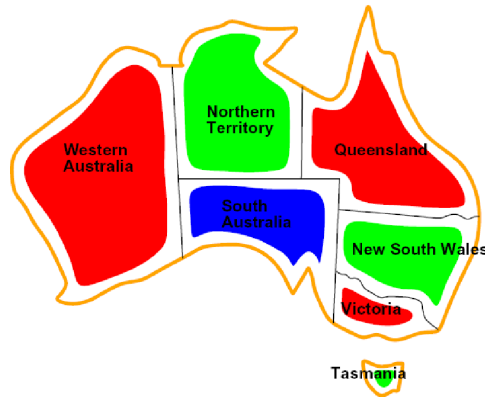
- Los CSP son una familia especializada de subproblemas de búsqueda.
- Idea principal: eliminar porciones grandes del espacio de búsqueda de una sola vez, identificando combinaciones de variable/valor que violan restricciones.

Formalmente, un **problema de satisfacción de restricciones** (CSP) consiste de tres componentes X , D y C :

- X es un conjunto de *variables*, $\{X_1, \dots, X_n\}$,
- D es un conjunto de *dominios*, $\{D_1, \dots, D_n\}$, uno para cada variable,
- C es un conjunto de *restricciones* que especifican las combinaciones de valores permitidas.

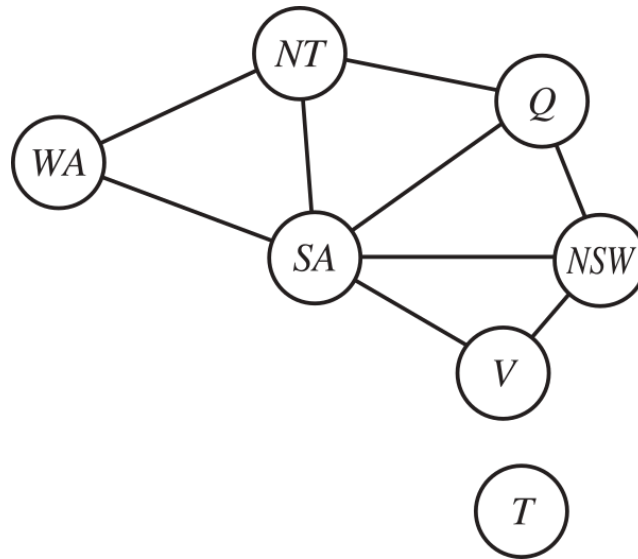
Ejemplo: coloreo de mapas





- Variables: $X = \{WA, NT, Q, NSW, V, SA, T\}$
- Dominios: $D_i = \{\text{rojo, verde, azul}\}$ para cada variable.
- Restricciones: $C = \{SA \neq WA, SA \neq NT, SA \neq Q, \dots\}$
 - Implícita: $WA \neq NT$
 - Explícita: $(WA, NT) \in \{\{\text{rojo, verde}\}, \{\text{rojo, azul}\}, \dots\}$
- Las soluciones son **asignaciones** de valores a las variables tales que se satisfacen todas las restricciones.
 - ej., $\{WA = \text{rojo}, NT = \text{verde}, Q = \text{rojo}, SA = \text{azul}, \quad NSW = \text{verde}, V = \text{rojo}, T = \text{verde}\}$

(Hiper)grafo de restricciones



- *Nodos* = variables del problema
- *Aristas* = restricciones del problema que involucran a las variables asociadas a los nodos extremos.
- Los algoritmos de CSP de propósito general

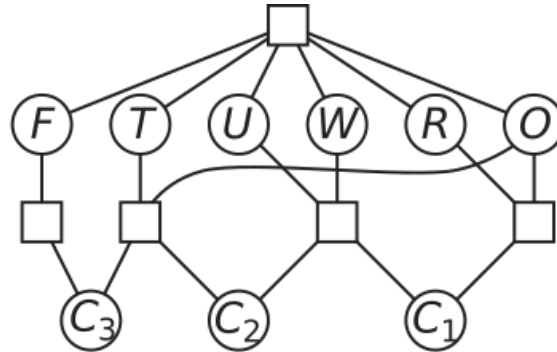
usan la estructura del grafo

para acelerar la búsqueda.

- ej., Tasmania es un subproblema independiente.

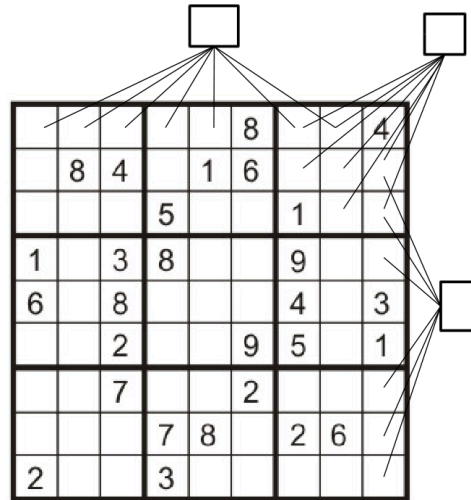
Ejemplo: criptoaritmética

$$\begin{array}{r} T W O \\ + T W O \\ \hline F O U R \end{array}$$

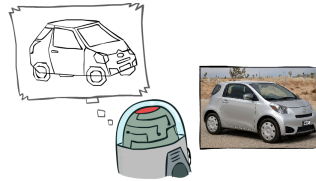


- Variables: $\{T, W, O, F, U, R, C_1, C_2, C_3\}$
- Dominios: $D_i = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
- Restricciones:
 - $\text{alldiff}(T, W, O, F, U, R)$
 - $O + O = R + 10 \times C_1$
 - $C_1 + W + W = U + 10 \times C_2$
 - ...

Ejemplo: Sudoku

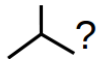
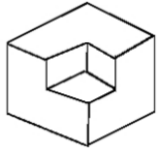


- Variables: cada casilla (abierta)
- Dominios: $D_i = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$
- Restricciones:
 - alldiff de 9 vías para cada columna
 - alldiff de 9 vías para cada fila
 - alldiff de 9 vías para cada región



Ejemplo: el algoritmo de Waltz

Procedimiento para interpretar dibujos 2D de líneas de poliedros sólidos como objetos 3D. Ejemplo temprano de un cómputo de IA planteado como un CSP.



Formulación como CSP:

- Cada intersección es una variable.
- Las intersecciones adyacentes se imponen restricciones entre sí.
- Las soluciones son objetos 3D físicamente realizables.

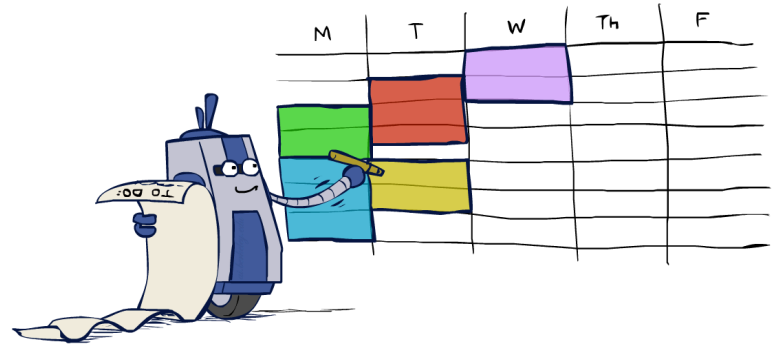
Variaciones sobre el formalismo CSP

- *Variables discretas*
 - Dominios finitos
 - Tamaño d implica $O(d^n)$ asignaciones completas.
 - ej., CSP booleanos, incluyendo el problema de satisfacibilidad booleana SAT (NP-completo).
 - Dominios infinitos
 - ej., programación de tareas, las variables son días de inicio/fin de cada tarea.
 - hace falta un lenguaje de restricciones, ej. $inicio_1 + 5 \leq inicio_2$.
 - Resoluble para restricciones lineales, indecidible en otro caso.
- *Variables continuas*
 - ej., tiempos precisos de inicio/fin de experimentos.
 - Las restricciones lineales son resolubles en tiempo polinomial con métodos de programación lineal.

- *Variedades de restricciones*
 - Las restricciones unarias involucran una sola variable.
 - Equivalente a reducir el dominio, ej. $SA \neq \text{verde}$.
 - Las restricciones binarias involucran pares de variables, ej. $SA \neq WA$.
 - Las restricciones de orden superior involucran 3 o más variables.
- *Preferencias* (restricciones suaves)
 - ej., rojo es mejor que verde.
 - Frecuentemente representables como un costo por cada asignación de variable.
 - Da lugar a problemas de optimización con restricciones.
 - (Las ignoramos en este curso.)

Ejemplos del mundo real

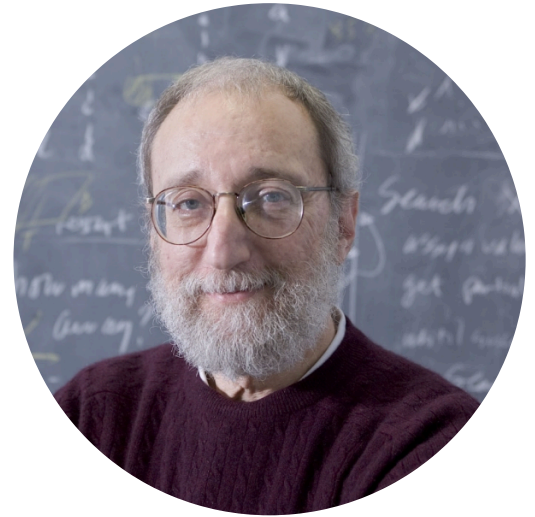
- Problemas de asignación
 - ej., quién enseña qué clase
- Problemas de programación de horarios
 - ej., ¿qué clase se ofrece cuándo y dónde?
- Configuración de hardware
- Hojas de cálculo
- Programación de transporte
- Programación de fábricas
- Diseño de circuitos
- ... y muchos más



Nota que muchos problemas del mundo real involucran variables de valor real.

Programación con restricciones

La programación con restricciones representa uno de los acercamientos más cercanos que la ciencia de la computación ha logrado al Santo Grial de la programación: el usuario plantea el problema, la computadora lo resuelve.
(Eugene Freuder)



La programación con restricciones es un paradigma de programación en el que el usuario especifica el programa como un CSP. La resolución del problema queda a cargo de la computadora.

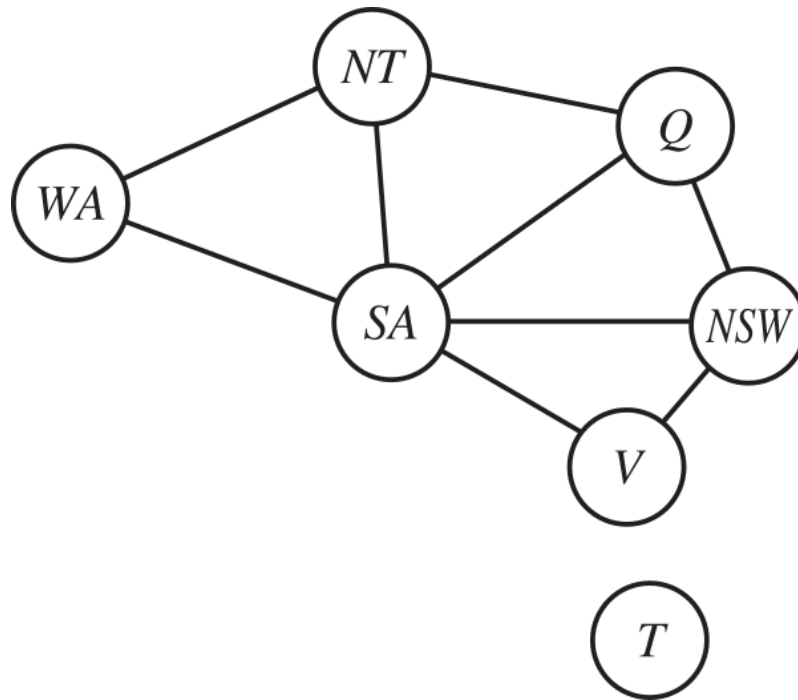
Ejemplos:

- Prolog
- ECLiPSe

Resolviendo CSP

Formulación como búsqueda estándar

- Los CSP se pueden plantear como problemas de búsqueda estándar.
 - Para los que ya tenemos solucionadores, incluyendo DFS, BFS o A*.
- Los estados son *asignaciones parciales*:
 - El estado inicial es la asignación vacía {}.
 - Acciones: asignar un valor a una variable sin asignar.
 - Prueba de objetivo: la asignación actual está completa y satisface todas las restricciones.
- ¡Este algoritmo es **el mismo** para todos los CSP!



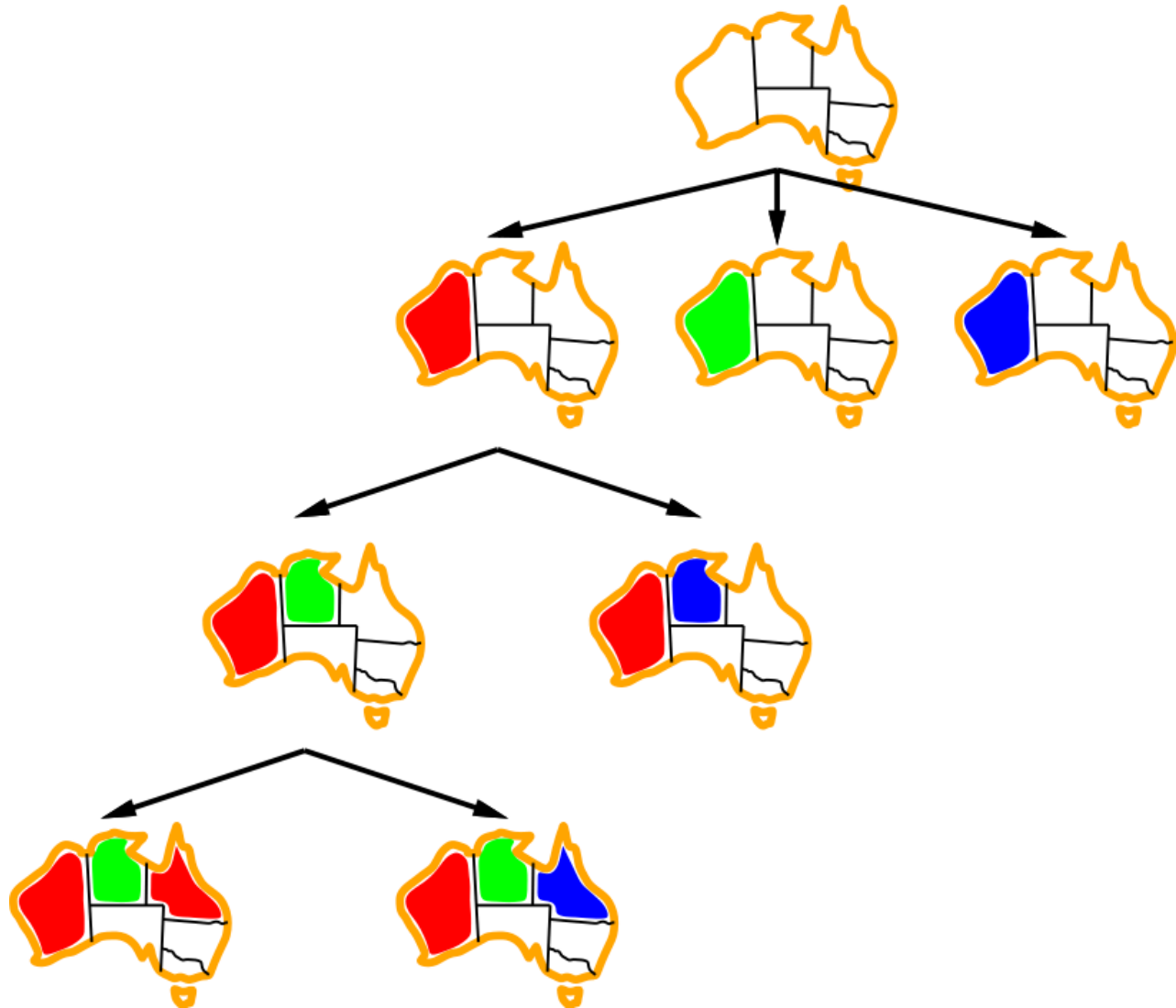
¿Qué haría BFS o DFS? ¿Qué problemas tiene la búsqueda ingenua?

Para n variables de dominio de tamaño d :

- $b = (n - l)d$ en la profundidad l ;
- ¡generamos un árbol con $n!d^n$ hojas aunque solo haya d^n asignaciones posibles!

Búsqueda con backtracking

- La búsqueda con backtracking es el algoritmo canónico no informado para resolver CSP.
- Idea 1: **una variable a la vez**:
 - La aplicación ingenua de algoritmos de búsqueda ignora una propiedad crucial: las asignaciones de variables son *conmutativas*. Por lo tanto, fijamos el orden.
 - $WA = \text{rojo}$ y luego $NT = \text{verde}$ es lo mismo que $NT = \text{verde}$ y luego $WA = \text{rojo}$.
 - Solo hace falta considerar asignaciones a una única variable en cada paso.
 - $b = d$ y hay d^n hojas.
- Idea 2: **revisar restricciones sobre la marcha**:
 - Considerar solo valores que no entren en conflicto con la asignación parcial actual.
 - Prueba de objetivo incremental.



```

function BACKTRACKING-SEARCH(csp) returns a solution, or failure
    return BACKTRACK({ }, csp)

function BACKTRACK(assignment, csp) returns a solution, or failure
    if assignment is complete then return assignment
    var  $\leftarrow$  SELECT-UNASSIGNED-VARIABLE(csp)
    for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
        if value is consistent with assignment then
            add { var = value } to assignment
            inferences  $\leftarrow$  INFERENCE(csp, var, value)
            if inferences  $\neq$  failure then
                add inferences to assignment
                result  $\leftarrow$  BACKTRACK(assignment, csp)
                if result  $\neq$  failure then
                    return result
            remove { var = value } and inferences from assignment
    return failure

```

Figure 6.5 A simple backtracking algorithm for constraint satisfaction problems. The algorithm is modeled on the recursive depth-first search of Chapter 3. By varying the functions SELECT-UNASSIGNED-VARIABLE and ORDER-DOMAIN-VALUES, we can implement the general-purpose heuristics discussed in the text. The function INFERENCE can optionally be used to impose arc-, path-, or k -consistency, as desired. If a value choice leads to failure (noticed either by INFERENCE or by BACKTRACK), then value assignments (including those made by INFERENCE) are removed from the current assignment and a new value is tried.

Mejorando el backtracking

¿Podemos mejorar el backtracking usando ideas de

propósito general

, sin conocimiento específico del dominio?

- *Ordenamiento*:
 - ¿Qué variable se debería asignar a continuación?
 - ¿En qué orden se deberían probar sus valores?
- *Filtrado*: ¿podemos detectar un fallo inevitable temprano?
- *Estructura*: ¿podemos explotar la estructura del problema?

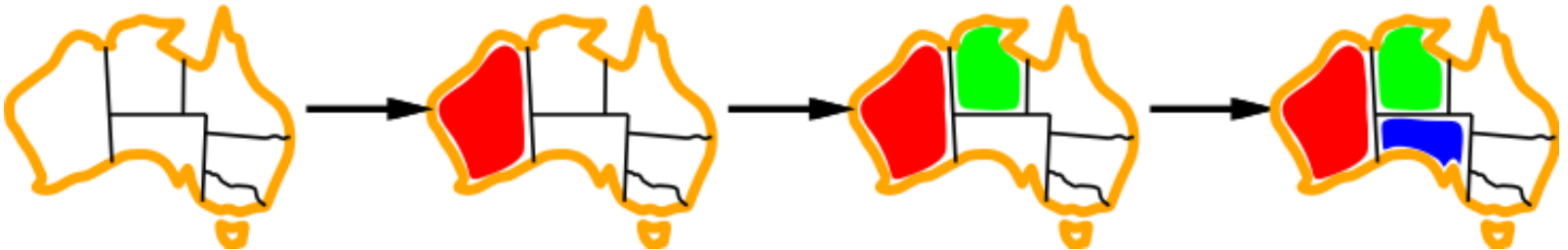
Ordenamiento de variables

- **Valores restantes mínimos (MRV):** elegir la variable

con menos valores legales restantes

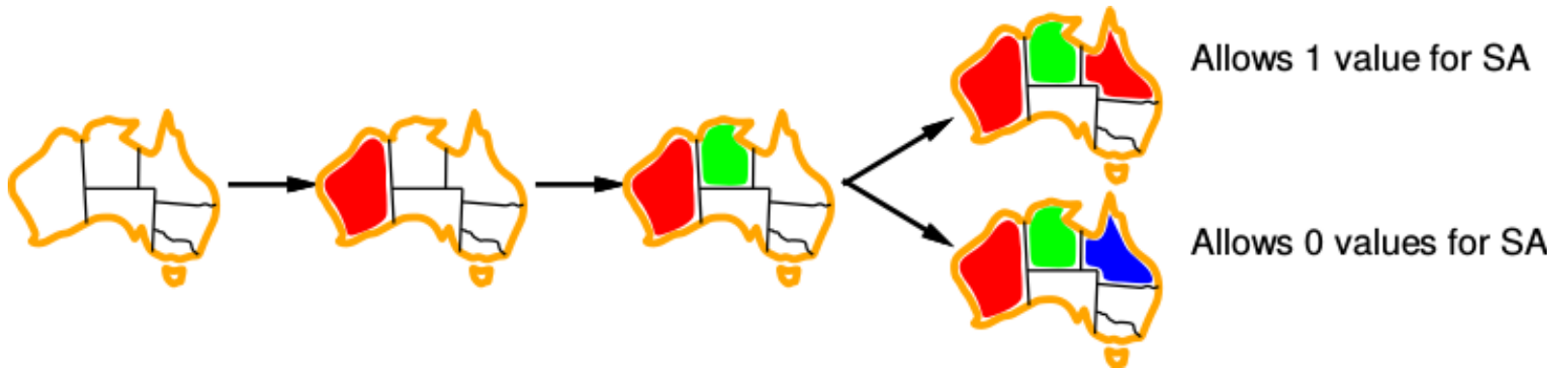
en su dominio.

- También conocida como la heurística de *fallar primero*.
 - Detectar fallos rápidamente equivale a podar partes grandes del árbol de búsqueda.



Ordenamiento de valores

- **Valor menos restrictivo (LCV):** dada una elección de variable, elegir el *valor menos restrictivo*.
- Es decir, el valor que descarta la menor cantidad de valores en las variables restantes.



¿Por qué la selección de variable debería fallar primero, pero la selección de valor debería fallar al final?

Filtrado: forward checking

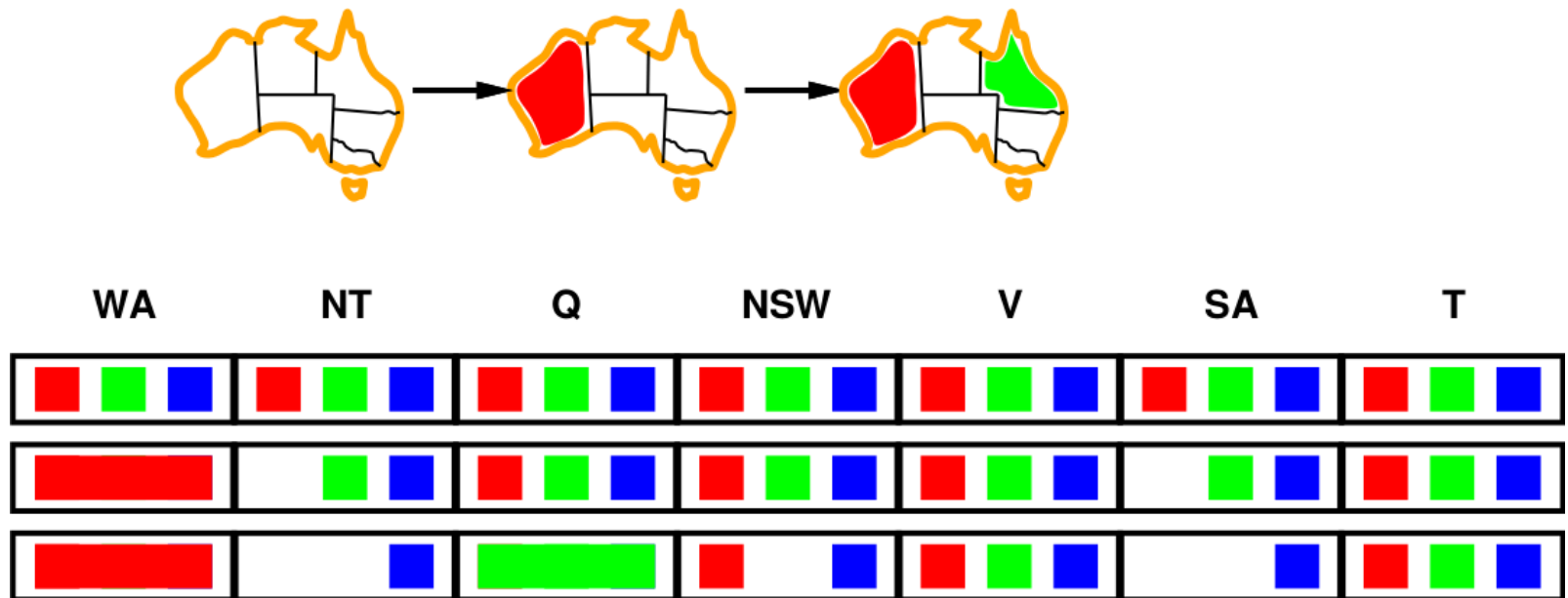
- Llevar el *registro de los valores legales restantes* para las variables sin asignar.
 - Cada vez que se asigna una variable X , y para cada variable sin asignar Y conectada a X por una restricción, eliminar del dominio de Y cualquier valor inconsistente.
- Terminar la *búsqueda* cuando alguna variable se quede sin valores legales.



WA	NT	Q	NSW	V	SA	T

Filtrado: propagación de restricciones

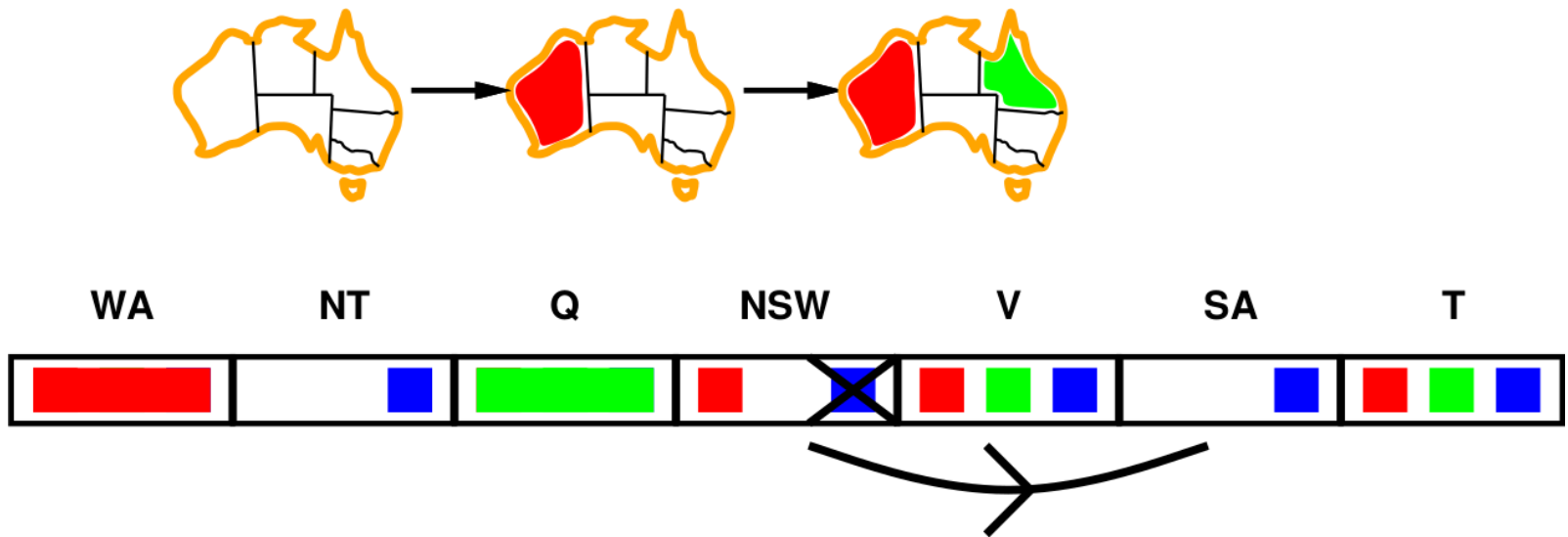
El forward checking propaga información hacia las variables sin asignar, pero no detecta tempranamente todos los fallos:



- ¡ NT y SA no pueden ser ambos azules!
- La **propagación de restricciones** refuerza restricciones localmente de forma repetida.

Consistencia de arcos

- Un arco $X \rightarrow Y$ es **consistente** si y solo si para todo valor x en el dominio de X existe algún valor y en el dominio de Y que satisface la restricción binaria asociada.
- Forward checking $\Leftrightarrow$ forzar la consistencia de los arcos que apuntan a cada nueva asignación.
- Este principio se puede generalizar para forzar la consistencia de **todos** los arcos.



function AC-3(*csp*) **returns** false if an inconsistency is found and true otherwise

inputs: *csp*, a binary CSP with components (X, D, C)

local variables: *queue*, a queue of arcs, initially all the arcs in *csp*

while *queue* is not empty **do**

$(X_i, X_j) \leftarrow \text{REMOVE-FIRST}(\textit{queue})$

if REVISE(*csp*, X_i, X_j) **then**

if size of $D_i = 0$ **then return** *false*

for each X_k **in** $X_i.\text{NEIGHBORS} - \{X_j\}$ **do**

 add (X_k, X_i) to *queue*

return *true*

function REVISE(*csp*, X_i, X_j) **returns** true iff we revise the domain of X_i

revised $\leftarrow$ *false*

for each x **in** D_i **do**

if no value y in D_j allows (x,y) to satisfy the constraint between X_i and X_j **then**

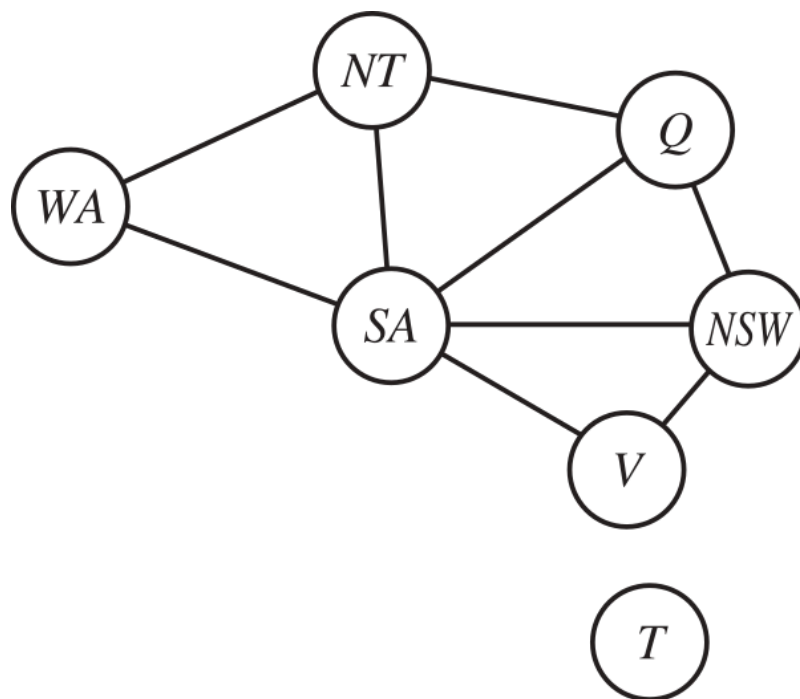
 delete x from D_i

revised $\leftarrow$ *true*

return *revised*

¿En qué momento del backtracking se debería llamar este procedimiento?

Estructura



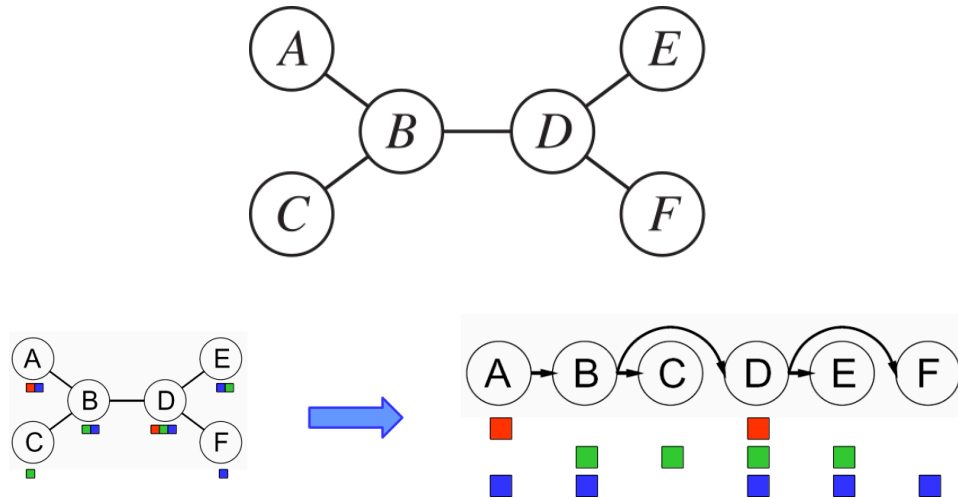
- Tasmania y el continente son **subproblemas independientes**.
 - Cualquier solución para el continente combinada con cualquier solución para Tasmania produce una solución para el mapa completo.
- La independencia se puede determinar encontrando las

Complejidad temporal

Supongamos que cada subproblema tiene c variables de n en total. Entonces $O(\frac{n}{c}d^c)$.

- ej., $n = 80, d = 2, c = 20$.
- $2^{80} = 4$ mil millones de años a 10 millones de nodos/seg.
- $4 \times 2^{20} = 0.4$ segundos a 10 millones de nodos/seg.

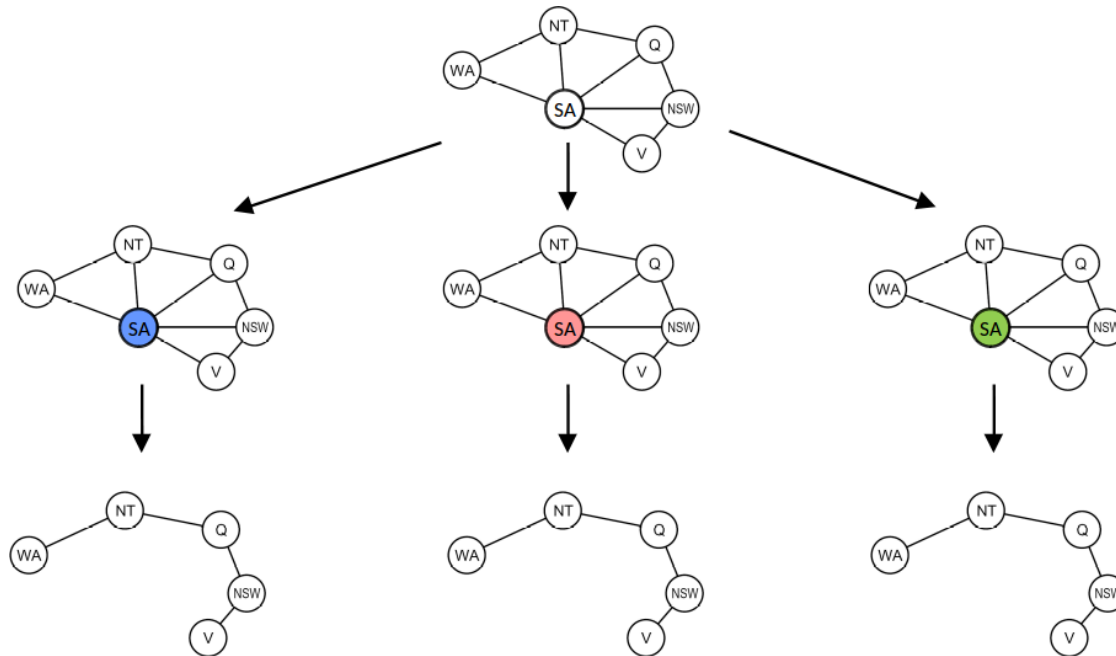
CSP con estructura de árbol



- Algoritmo para CSP con estructura de árbol:
 - Ordenar: elegir una variable raíz, ordenar las variables de modo que los padres precedan a los hijos (orden topológico).
 - Eliminar hacia atrás:
 - para $i = n$ hasta 2 , forzar la consistencia de arcos de $padre(X_i) \rightarrow X_i$.
 - Asignar hacia adelante:
 - para $i = 1$ hasta n , asignar X_i de forma consistente con su $padre(X_i)$.
- Complejidad temporal: $O(nd^2)$
 - En comparación con los CSP generales, donde el peor caso es $O(d^n)$.

CSP casi con estructura de árbol

- *Condicionamiento*: instanciar una variable, podar los dominios de sus vecinos.
- *Condicionamiento por conjunto de corte (cutset)*:
 - Asignar (de todas las formas posibles) un conjunto S de variables tal que el grafo de restricciones restante sea un árbol.
 - Resolver los CSP residuales (con estructura de árbol).
 - Si el CSP residual tiene solución, devolverla junto con la asignación para S .



Resumen

- Los estados de un CSP se representan con un conjunto de pares variable/valor, no como estados atómicos.
- El backtracking, una forma de búsqueda en profundidad, se usa comúnmente para resolver CSP — es el mismo algoritmo para todos los CSP.
- El ordenamiento (MRV, LCV) y el filtrado (forward checking, consistencia de arcos) aceleran el backtracking sin conocimiento específico del dominio.
- La complejidad de resolver un CSP está fuertemente relacionada con la estructura de su grafo de restricciones — los CSP con estructura de árbol se resuelven en tiempo lineal.

Fin de la Clase 3

Próxima clase: Juegos adversariales — minimax y poda alfa-beta